

List vs. Set

The Java `List` and **Java Set** interfaces are quite similar in that they both represent a collection of elements. However, there are some significant differences. These differences are reflected in the methods the `List` and `Set` interfaces offer.

The first difference between the Java `List` and Java `Set` interface is, that the same element can occur more than once in a Java `List`. This is different from a Java `Set` where each element can occur only once.

The second difference between a Java `List` and Java `Set` interfaces is, that the elements in a `List` has an order, and the elements can be iterated in that order. A Java `Set` does not make any promises about the order of the elements kept internally.

List Implementations

Being a `Collection` subtype all methods in the `Collection` interface are also available in the `List` interface.

Since `List` is an interface you need to instantiate a concrete implementation of the interface in order to use it. You can choose between the following `List` implementations in the Java Collections API:

- `java.util.ArrayList`
- `java.util.LinkedList`
- `java.util.Vector`
- `java.util.Stack`

Of these implementations, the `ArrayList` is the most commonly used.

There are also concurrent `List` implementations in the `java.util.concurrent` package. These `List` implementations are explained in more detail in my **[java.util.concurrent tutorial](#)** .

Create a List

You create a `List` instance by creating an instance of one of the classes that implements the `List` interface. Here are a few examples of how to create a `List` instance:

```
List listA = new ArrayList();
List listB = new LinkedList();
List listC = new Vector();
List listD = new Stack();
```

Remember, most often you will use the `ArrayList` class, but there can be cases where using one of the other implementations might make sense.

Generic Lists

By default you can put any `Object` into a `List`, but from Java 5, Java Generics makes it possible to limit the types of object you can insert into a `List`. Here is an example:

```
List<MyObject> list = new ArrayList<MyObject>();
```

This `List` can now only have `MyObject` instances inserted into it. You can then access and iterate its elements without casting them. Here is how it looks:

```
List<MyObject> list = new ArrayList<MyObject>();

list.add(new MyObject("First MyObject"));

MyObject myObject = list.get(0);

for(MyObject anObject : list){
    //do something to anObject...
}
```

Without generics the example above would have looked like this:

```
List list = new ArrayList();    //no generic type specified

list.add(new MyObject("First MyObject"));

MyObject myObject = (MyObject) list.get(0);    //cast needed

for(Object anObject : list){
    //cast needed
    MyObject theMyObject = (MyObject) anObject;
```

```
//do something to anObject...  
}
```

Notice how it is necessary to cast the `MyObject` instances retrieved from the `List` to `MyObject`. Without a generic type set on the `List` variable declaration the Java compiler only knows that the `List` contains `Object` instances. Thus, you need to cast them to the concrete class (or interface) that you know the object to be of.

It is a good practice to specify a generic type for your `List` variables whenever you can. It helps you avoid inserting the wrong types of objects into the `List`. It enables you to retrieve the objects from the `List` without having to cast them their real type. And - it helps the reader of your code see what type of objects the `List` is supposed to contain. You should only omit a generic type if you have very good reasons to do so.

Throughout the rest of this Java `List` tutorial I will be using generic `List` examples as much as possible.

For more information about Java Generics, see the [Java Generics Tutorial](#).

Insert Elements in a Java List

You insert elements (objects) into a Java `List` using its `add()` method. Here is an example of adding elements to a Java `List` using the `add()` method:

```
List<String> listA = new ArrayList<>();  
  
listA.add("element 1");  
listA.add("element 2");  
listA.add("element 3");
```

The first three `add()` calls add a `String` instance to the end of the list.

Insert null Values

It is actually possible to insert `null` values into a Java `List`. Here is an example of inserting a `null` value into a Java `List`:

```
Object element = null;
```

```
List<Object> list = new ArrayList<>();  
  
list.add(element);
```

Insert Elements at Specific Index

It is possible to insert an element into a Java `List` at a specific index.

The `List` interface has a version of the `add()` method that takes an index as first parameter, and the element to insert as the second parameter. Here is an example of inserting an element at index 0 into a Java `List`:

```
list.add(0, "element 4");
```

If the `List` already contained elements, these elements will now get pushed further down in the `List`'s internal sequence. The element that had index 0 before the new element was inserted at index 0, will get pushed to index 1 etc.

Insert All Elements From One List Into Another

It is possible to add all elements from one Java `List` into another `List`. You do so using the `List` `addAll()` method. The resulting `List` is the *union* of the two lists. Here is an example of adding all elements from one `List` into another:

```
List<String> listSource = new ArrayList<>();  
  
listSource.add("123");  
listSource.add("456");  
  
List<String> listDest    = new ArrayList<>();  
  
listDest.addAll(listSource);
```

This example adds all elements from `listSource` into `listDest`.

The `addAll()` method takes a `Collection` as parameter, so you can pass either a `List` or **Java Set** as parameter. In other words, you can add all elements from a `List` or `Set` into a `List` with `addAll()`.

Get Elements From a Java List

You can get the elements from a Java `List` using the index of the elements. You can do so using either the `get(int index)` method. Here is an example of accessing the elements of a Java `List` using the element indexes:

```
List<String> listA = new ArrayList<>();

listA.add("element 0");
listA.add("element 1");
listA.add("element 2");

//access via index
String element0 = listA.get(0);
String element1 = listA.get(1);
String element3 = listA.get(2);
```

It is also possible to iterate the elements of a Java `List` in the order they are stored in internally. I will show you how to do that later in this Java List tutorial.

Find Elements in a List

You can find elements in a Java `List` using one of these two methods:

- `indexOf()`
- `lastIndexOf()`

The `indexOf()` method finds the index of the first occurrence in the `List` of the given element. Here is an example finding the index of two elements in a Java `List`:

```
List<String> list = new ArrayList<>();

String element1 = "element 1";
String element2 = "element 2";

list.add(element1);
list.add(element2);

int index1 = list.indexOf(element1);
int index2 = list.indexOf(element2);

System.out.println("index1 = " + index1);
System.out.println("index2 = " + index2);
```

Running this code will result in this output:

```
index1 = 0
index2 = 1
```

Find Last Occurrence of Element in a List

The `lastIndexOf()` method finds the index of the last occurrence in the `List` of a given element. Here is an example showing how to find the index of the last occurrence of a given element in a Java `List`:

```
List<String> list = new ArrayList<>();

String element1 = "element 1";
String element2 = "element 2";

list.add(element1);
list.add(element2);
list.add(element1);

int lastIndex = list.lastIndexOf(element1);
System.out.println("lastIndex = " + lastIndex);
```

The output printed from running the above Java example will be:

```
lastIndex = 2
```

The element `element 1` occurs 2 times in the `List`. The index of the last occurrence is 2.

Checking if List Contains Element

You can check if a Java `List` contains a given element using the `List contains()` method. Here is an example of checking if a Java `List` contains an element using the `contains()` method:

```
List<String> list = new ArrayList<>();

String element1 = "element 1";

list.add(element1);

boolean containsElement =
    list.contains("element 1");
```

```
System.out.println(containsElement);
```

The output from running this Java `List` example will be:

```
true
```

... because the `List` does actually contain the element.

To determine if the `List` contains the element, the `List` will internally iterate its elements and compare each element to the object passed as parameter. The comparison uses the **Java equals** method of the element to check if the element is equal to the parameter.

Since it is possible to add `null` values to a `List`, it is actually possible to check if the `List` contains a `null` value. Here is how you check if a `List` contains a `null` value:

```
list.add(null);  
  
containsElement = list.contains(null);  
  
System.out.println(containsElement);
```

Obviously, if the input parameter to `contains()` is `null`, the `contains()` method will not use the `equals()` method to compare against each element, but rather use the `==` operator.

Remove Elements From a Java List

You can remove elements from a Java `List` via these two methods:

1. `remove(Object element)`
2. `remove(int index)`

`remove(Object element)` removes that element in the list, if it is present. All subsequent elements in the list are then moved up in the list. Their index thus decreases by 1. Here is an example of removing an element from a Java `List` based on the element itself:

```
List<String> list = new ArrayList<>();
```

```
String element = "first element";  
list.add(element);  
  
list.remove(element);
```

This example first adds an element to a `List` and then removes it again.

The `List remove(int index)` method removes the element at the given index. All subsequent elements in the list are then moved up in the list. Their index thus decreases by 1. Here is an example of removing an element from a Java `List` by its index:

```
List<String> list = new ArrayList<>();  
  
list.add("element 0");  
list.add("element 1");  
list.add("element 2");  
  
list.remove(0);
```

After running this Java example code the `List` would contain the **Java String** elements `element 1` and `element 2` at index 0 and 1. The first element (`element 0`) has been removed from the `List`.

Remove All Elements From a Java List

The Java `List` interface contains a `clear()` method which removes all elements from the list when called. Removing all elements from a `List` is also referred to as *clearing* a `List`. Here is simple example of removing all elements from a `List` (clearing) with the `clear()` method:

```
List<String> list = new ArrayList<>();  
  
list.add("object 1");  
list.add("object 2");  
//etc.  
  
list.clear();
```

First, a new `List` is created. Second, two elements are added to the `List`. Third, the `clear()` method is called. After the `clear()` method has been called, the `List` will be completely empty.

Retain All Elements From One List in Another

The Java `List` interface has a method called `retainAll()` which is capable of retaining all elements from one `List` which are also present in another `List`. In other words, the `retain()` method removes all the elements from the target `List` which are not found in the other `List`. The resulting `List` is the *intersection* of the two lists. Here is a Java example of calling the `List retainAll()` method:

```
List<String> list      = new ArrayList<>();
List<String> otherList = new ArrayList<>();

String element1 = "element 1";
String element2 = "element 2";
String element3 = "element 3";
String element4 = "element 4";

list.add(element1);
list.add(element2);
list.add(element3);

otherList.add(element1);
otherList.add(element3);
otherList.add(element4);

list.retainAll(otherList);
```

First two lists are created. Second, 3 elements are added to `list` and 3 elements are added to `otherList`. Third, the `retainAll()` method is called on `list`, passing `otherList` as parameter. After `list.retainAll(otherList)` has finished executing, the `list` will only contain those elements which were both present in `list` and `otherList` before `retainAll()` was called. More specifically, that is `element1` and `element3`.

List Size

You can obtain the number of elements in the `List` by calling the `size()` method. Here is an example:

```
List<String> list = new ArrayList<>();

list.add("object 1");
list.add("object 2");

int size = list.size();
```

Sublist of List

The Java `List` interface has a method called `subList()` which can create a new `List` with a subset of the elements from the original `List`.

The `subList()` method takes 2 parameters: A start index and an end index. The start index is the index of the first element from the original `List` to include in the sublist. The end index is the last index of the sublist, but the element at the last index is not included in the sublist. This is similar to how the **Java String substring** method works.

Here is a Java example of creating a sublist of elements from another `List` using the `subList()` method:

```
List<String> list = new ArrayList<>();

list.add("element 1");
list.add("element 2");
list.add("element 3");
list.add("element 4");

List<String> sublist = list.subList(1, 3);
```

After executing the `list.subList(1,3)` instruction the `sublist` will contain the elements at index 1 and 2. Remember, the original list has 4 elements with indexes from 0 to 3. The `list.subList(1,3)` call will include index 1, but exclude index 3, thereby keeping the elements at index 1 and 2.

Convert List to Set

You can convert a Java `List` to a Java `Set` by creating a new `Set` and adding all elements from the `List` to it. The `Set` will remove all duplicates. Thus, the resulting `Set` will contain all the elements from the `List`, but only once. Here is an example of converting a Java `List` to a `Set`:

```
List<String> list = new ArrayList<>();

list.add("element 1");
list.add("element 2");
list.add("element 3");
list.add("element 3");

Set<String> set = new HashSet<>();
```

```
set.addAll(list);
```

Notice that the `List` contains the `String` element 3 two times. The `Set` will only contain that `String` once. Thus, the resulting set will contain the `Strings` element 1, element 2 and element 3.

Convert List to Array

You can convert a Java `List` to a **Java Array** using the `List toArray()` method. Here is an example of converting a Java `List` to a Java array:

```
List<String> list = new ArrayList<>();

list.add("element 1");
list.add("element 2");
list.add("element 3");
list.add("element 3");

Object[] objects = list.toArray();
```

It is also possible to convert a `List` to an array of a specific type. Here is an example of converting a Java `List` to an array of a specific type:

```
List<String> list = new ArrayList<>();

list.add("element 1");
list.add("element 2");
list.add("element 3");
list.add("element 3");

String[] objects1 = list.toArray(new String[0]);
```

Note that even if we pass a `String` array of size 0 to the `toArray()`, the array returned will have all the elements in the `List` in it. It will have the same number of elements as the `List`.

Convert Array to List

It is also possible to convert a Java `List` to an array. Here is an example of converting a Java array to a `List`:

```
String[] values = new String[]{ "one", "two", "three" };  
  
List<String> list = (List<String>) Arrays.asList(values);
```

It is the `Arrays.asList()` method that converts the array to a `List`.

Sort List

You can sort a Java `List` using the `Collections.sort()` method. I have explained how that works in my [sorting Java collections tutorial](#), but I will just show you a few ways to sort a Java `List` in the following sections.

Sort List of Comparable Objects

If the `List` contains objects that implement the `Comparable` interface (`java.lang.Comparable`), then the objects can compare themselves to each other. In that case you can sort the `List` like this:

```
List<String> list = new ArrayList<>();  
  
list.add("c");  
list.add("b");  
list.add("a");  
  
Collections.sort(list);
```

The Java `String` class implements the `Comparable` interface, you can sort them in their natural order, using the `Collections.sort()` method.

Sort List Using Comparator

If the objects in the Java `List` do not implement the `Comparable` interface, or if you want to sort the objects in another order than their `compare()` implementation, then you need to use a `Comparator` implementation (`java.util.Comparator`). Here is an example of sorting a list of `Car` objects using a `Comparator`. Here is first the `Car` class:

```
public class Car{  
    public String brand;  
    public String numberPlate;
```

```

    public int noOfDoors;

    public Car(String brand, String numberPlate, int noOfDoors) {
        this.brand = brand;
        this.numberPlate = numberPlate;
        this.noOfDoors = noOfDoors;
    }
}

```

Here is the code that sorts a Java `List` of the above `Car` objects:

```

List<Car> list = new ArrayList<>();

list.add(new Car("Volvo V40" , "XYZ 201845", 5));
list.add(new Car("Citroen C1", "ABC 164521", 4));
list.add(new Car("Dodge Ram" , "KLM 845990", 2));

Comparator<Car> carBrandComparator = new Comparator<Car>() {
    @Override
    public int compare(Car car1, Car car2) {
        return car1.brand.compareTo(car2.brand);
    }
};

Collections.sort(list, carBrandComparator);

```

Notice the `Comparator` implementation in the example above. This implementation only compares the `brand` field of the `Car` objects. It is possible to create another `Comparator` implementation which compares the number plates, or even the number of doors in the cars.

Note too, that it is possible to implement a `Comparator` using a **Java Lambda**. Here is an example that sorts a `List` of `Car` objects using three different Java lambda implementations of the `Comparator` interface, each of which compares `Car` instances by a different field:

```

List<Car> list = new ArrayList<>();

list.add(new Car("Volvo V40" , "XYZ 201845", 5));
list.add(new Car("Citroen C1", "ABC 164521", 4));
list.add(new Car("Dodge Ram" , "KLM 845990", 2));

Comparator<Car> carBrandComparatorLambda =
    (car1, car2) -> car1.brand.compareTo(car2.brand);

Comparator<Car> carNumberPlatComparatorLambda =
    (car1, car2) -> car1.numberPlate.compareTo(car2.numberPlate);

Comparator<Car> carNoOfDoorsComparatorLambda =

```

```
(car1, car2) -> car1.noOfDoors - car2.noOfDoors;

Collections.sort(list, carBrandComparatorLambda);
Collections.sort(list, carNumberPlatComparatorLambda);
Collections.sort(list, carNoOfDoorsComparatorLambda);
```

Iterate List

You can iterate a Java `List` in several different ways. The three most common ways are:

- Using an `Iterator`
- Using a for-each loop
- Using a for loop
- Using the Java Stream API

I will explain each of these methods of iterating a Java `List` in the following sections.

Iterate List Using Iterator

The first way to iterate a `List` is to use a **Java Iterator**. Here is an example of iterating a `List` with an `Iterator`:

```
List<String> list = new ArrayList<>();

list.add("first");
list.add("second");
list.add("third");

Iterator<String> iterator = list.iterator();
while(iterator.hasNext()) {
    String next = iterator.next();
}
```

You obtain an `Iterator` by calling the `iterator()` method of the `List` interface.

Once you have obtained an `Iterator` you can keep calling its `hasNext()` method until it returns `false`. Calling `hasNext()` is done inside a `while` loop as you can see.

Inside the `while` loop you call the `Iterator next()` method of the `Iterator` interface to obtain the next element pointed to by the `Iterator`.

If the `List` is typed using **Java Generics** you can save some object casting inside the `while` loop. Here is an example:

```
List<String> list = new ArrayList<>();

list.add("first");
list.add("second");
list.add("third");

Iterator<String> iterator = list.iterator();
while(iterator.hasNext()){
    String obj = iterator.next();
}
```

Iterate List Using For-Each Loop

The second way to iterate a `List` is to use the `for` loop added in Java 5 (also called a "for each" loop). Here is an example of iterating a `List` using the `for` loop:

```
List list = new ArrayList();

list.add("first");
list.add("second");
list.add("third");

for(Object element : list) {
    System.out.println(element);
}
```

The `for` loop is executed once per element in the `List`. Inside the `for` loop each element is in turn bound to the `obj` variable.

If the list is typed (a *generic* `List`) you can change the type of the variable inside the `for` loop. Here is typed `List` iteration example:

```
List<String> list = new ArrayList<String>();

//add elements to list

for(String element : list) {
    System.out.println(element);
}
```

Notice how the `List` is typed to `String`. Therefore you can set the type of the variable inside the `for` loop to `String`.

Iterate List Using For Loop

The third way to iterate a `List` is to use a standard `for` loop like this:

```
List list = new ArrayList();

list.add("first");
list.add("second");
list.add("third");

for(int i=0; i < list.size(); i++) {
    Object element = list.get(i);
}
```

The `for` loop creates an `int` variable and initializes it to 0. Then it loops as long as the `int` variable `i` is less than the size of the list. For each iteration the variable `i` is incremented.

Inside the `for` loop the example accesses the elements in the `List` via its `get()` method, passing the incrementing variable `i` as parameter.

Again, if the `List` is typed using Java Generics to e.g. to a `String`, then you can use the generic type of the `List` as type for the local variable that is assigned each element in the `List` during iteration. An example will make this more clear:

```
List<String> list = new ArrayList<String>();

list.add("first");
list.add("second");
list.add("third");

for(int i=0; i < list.size(); i++) {
    String element = list.get(i);
}
```

Notice the type of the local variable inside the `for` loop is now `String`. Because the `List` is generically typed to `String`, it can only contain `String` objects. Hence, the compiler knows that only a `String` can be returned from the `get()` method. Therefore you do not need to cast the element returned by `get()` to `String`.

Iterate List Using Java Stream API

The fourth way to iterate a Java `List` is via the **Java Stream** API. To iterate a Java `List` you must first obtain a `Stream` from the `List`. Obtaining a `Stream` from

a `List` in Java is done by calling the `List stream()` method. Here is an example of obtaining a Java `Stream` from a Java `List`:

```
List<String> stringList = new ArrayList<String>();

stringList.add("abc");
stringList.add("def");

Stream<String> stream = stringList.stream();
```

It is the last line of this example that calls the `List stream()` method to obtain the `Stream` representing the elements in the `List`.

Once you have obtained a `Stream` from a `List` you can iterate the `Stream` by calling its `forEach()` method. Here is an example of iterating the elements of a `List` using the `Stream forEach()` method:

```
List<String> stringList = new ArrayList<String>();

stringList.add("one");
stringList.add("two");
stringList.add("three");

Stream<String> stream = stringList.stream();
stream
    .forEach( element -> { System.out.println(element); } );
```

Calling the `forEach()` method will make the `Stream` iterate all the element of the `Stream` internally, and call the `Consumer` passed as parameter to the `forEach()` method for each element in the `Stream`.